

POLITECHNIKA RZESZOWSKA
KATEDRA INFORMATYKI I AUTOMATYKI
DR INŻ. MATEUSZ POMIANEK

ĆWICZENIE LABORATORYJNE FLUTTER 3

Publikacja aplikacji · CI/CD · Flavors ·

Programowanie Aplikacji Mobilnych - moduł Flutter

Projekt: WeatherApp v3 - rozszerzenie projektu z Flutter Lab 2

Wymagania wstępne: ukończony Flutter Lab 2 (Riverpod, Hive, animacje, testy widgetów)

Spis treści

1. Czym jest Flavor i dlaczego każda aplikacja go potrzebuje?	1
2. Konfiguracja Flavor-ów w Flutter	2
3. Podpisywanie aplikacji, czyli Keystore i upload key	6
4. Budowanie artefaktów; APK vs App Bundle	7
5. Od konta dewelopera do publikacji na Google Play Store	8
6. Automatyzacja potoku, CI/CD z GitHub Actions	9
7. Automatyzacja publikacji z Fastlane	12
8. Zarządzanie sekretami i zmiennymi środowiskowymi	14
9. Monitorowanie przez Firebase Crashlytics i Analytics	15

1. Czym jest Flavor i dlaczego każda aplikacja go potrzebuje?

Wyobraź sobie, że pracujesz nad WeatherApp. Masz dwie wersje: jedna łączy się z prawdziwym API pogodowym (produkcja), druga z serwerem testowym (development). No i łatwo tu o problem. Jak jednocześnie mieć obie na telefonie, nie mylić kluczy API i nie wypychać kodu testowego do użytkowników?

Odpowiedzią są Flavors (znane również jako Product Flavors w świecie Androida lub Build Configurations w iOS/Xcode). To mechanizm budowania wielu wariantów tej samej aplikacji z jednej bazy kodu.

Analogia jako kuchnia restauracyjna

Wyobraź sobie restaurację, która serwuje ten sam przepis na zupę w trzech wersjach: wegetariańskiej, mięsnej i bezglutenowej. Składniki są podobne, ale kilka kluczowych się różni. Szef kuchni nie pisze trzech odrębnych przepisów - ma jeden bazowy i adnotuje różnice. Flavors w Flutterze działają identycznie: jeden kod bazowy, kilka zestawów konfiguracji.

1.1. Trzy środowiska aplikacji mobilnej

Profesjonalna aplikacja mobilna zwykle ma trzy środowiska. Każde ma inne: URL API, klucze dostępowe, ustawienia logowania i identyfikator pakietu.

Flavor / środowisko	Cel	Cechy charakterystyczne	Pakiet (Android)
development (dev)	Aktywny development, testy funkcji	Ładny banner DEBUG, szczegółowe logi, mock lub testowe API, HMR/Hot Reload	<i>pl.edu.pam.weather.dev</i>
staging	Testy QA, testy akceptacyjne klienta	Produkcyjny wygląd, testowe API, dane testowe, instalacja obok prod	<i>pl.edu.pam.weather.staging</i>
production (prod)	Wersja dla użytkowników w Play Store	Brak logów debug-owych, prawdziwe API, zaciemniony kod (obfuscation)	<i>pl.edu.pam.weather</i>

Dlaczego różne identyfikatory pakietów?

Na urządzeniu Android mogą współistnieć dwie aplikacje o tym samym nazewnictwie tylko wtedy, gdy mają różne package names. Używając *pl.edu.pam.weather.dev* i *pl.edu.pam.weather* możesz jednocześnie testować obie wersje na jednym telefonie.

1.2. Co Flavor zmienia, a co pozostaje takie samo?

Co zmienia Flavor	Co pozostaje IDENTYCZNE
Identyfikator pakietu (package name / bundle ID)	Cały kod Dart (logika, widgety, UI)
Nazwa aplikacji wyświetlana pod ikoną	Zależności w pubspec.yaml
Ikonka aplikacji i splash screen	Struktura katalogów projektu
Klucz API, URL backendu, klucze Firebase	Testy jednostkowe i widжетów
Podpisywanie (keystore / upload key)	Logika biznesowa, modele danych
Poziom logowań, sentry DSN, analytics key	Paczki / pluginy Flutter

2. Konfiguracja Flavor-ów w Flutter

Konfiguracja Flavorów we Flutterze dzieli się na dwie części: stronę Dart (punkt wejścia aplikacji, klasa konfiguracji) i stronę natywną (Android Gradle, iOS Xcode schemes). Zaczniemy od strony Dart, bo ona kontroluje logikę, a nie tylko budowanie.

2.1. Klasa konfiguracji AppConfig

Zamiast hardkodować URL API i klucze w kodzie, tworzymy klasę AppConfig, która przechowuje wszystkie wartości zależne od środowiska. Jest inicjalizowana PRZED uruchomieniem aplikacji.

Wzorzec

Singleton z inicjalizacją przed runApp(). AppConfig jest inicjalizowany raz w main(), zanim Flutter uruchomi drzewo widжетów. Wszystkie serwisy pobierają konfigurację z AppConfig.instance.

Actions, Flavors

// lib/core/config/app_config.dart

enum AppFlavor { dev, staging, prod }

```

class AppConfig {
  // Singleton - jedna instancja na caly czas zycia aplikacji
  static AppConfig? _instance;
  static AppConfig get instance {
    assert(_instance != null, 'AppConfig nie zostal zainicjalizowany!');
    return _instance!;
  }

  // Pola konfiguracji
  final AppFlavor flavor;
  final String baseUrl;
  final String appName;
  final bool enableAnalytics;
  final bool showDebugBanner;
  final String sentryDsn;

  AppConfig._({
    required this.flavor,
    required this.baseUrl,
    required this.appName,
    required this.enableAnalytics,
    required this.showDebugBanner,
    this.sentryDsn = '',
  });

  // Gettery pomocnicze
  bool get isDev      => flavor == AppFlavor.dev;
  bool get isStaging  => flavor == AppFlavor.staging;
  bool get isProd     => flavor == AppFlavor.prod;

  // Fabryki dla kazdego flavoru
  static void initDev() {
    _instance = AppConfig._(
      flavor: AppFlavor.dev,
      baseUrl: 'https://api-dev.openmeteo.test',
      appName: '[DEV] WeatherApp',
      enableAnalytics: false,      // brak analityk w dev
      showDebugBanner: true,      // czerwony banner DEBUG
    );
  }

  static void initStaging() {
    _instance = AppConfig._(
      flavor: AppFlavor.staging,
      baseUrl: 'https://api-staging.openmeteo.test',
      appName: '[STG] WeatherApp',
      enableAnalytics: false,
      showDebugBanner: false,
    );
  }

  static void initProd() {
    _instance = AppConfig._(
      flavor: AppFlavor.prod,
      baseUrl: 'https://api.open-meteo.com',      // prawdziwe API
      appName: 'WeatherApp',
      enableAnalytics: true,                      // analytics wlaczone
      showDebugBanner: false,
      sentryDsn: 'https://xxx@sentry.io/yyy',    // monitoring błędów
    );
  }
}

```

2.2. Osobne punkty wejścia:- main_*.dart

Kazdy flavor ma swój plik main_*.dart. To jest punkt wejścia uruchamiany przez flutter run --flavor. Różnica między plikami jest minimalna - wywołanie innej metody init().

// lib/main_dev.dart - punkt wejścia dla flavoru dev

```
import 'package:flutter/material.dart';  
import 'core/config/app_config.dart';  
import 'app.dart'; // wspólna klasa MyApp
```

```
void main() {  
  AppConfig.initDev();    // ZAWSZE przed runApp()  
  runApp(const MyApp());  
}
```

// lib/main_staging.dart

```
void main() { AppConfig.initStaging(); runApp(const MyApp()); }
```

// lib/main_prod.dart (lub main.dart dla kompatybilności)

```
void main() { AppConfig.initProd(); runApp(const MyApp()); }
```

// lib/app.dart - wspólna klasa MyApp (nie zmienia się między flavorami!)

```
class MyApp extends ConsumerWidget {  
  const MyApp({super.key});
```

```
@override
```

```
Widget build(BuildContext context, WidgetRef ref) {  
  return MaterialApp.router(  
    title: AppConfig.instance.appName,  
    debugShowCheckedModeBanner: AppConfig.instance.showDebugBanner,  
    theme: AppTheme.lightTheme,  
    darkTheme: AppTheme.darkTheme,  
    themeMode: ThemeMode.system,  
    routerConfig: appRouter,  
  );  
}
```

2.3. Konfiguracja Android - Gradle

Strona Dart to tylko połowa pracy. Android musi wiedzieć o flavorach, aby nadać im różne package names, ikony i nazwy. Konfiguracja w android/app/build.gradle.

```
// android/app/build.gradle
```

```
android {  
  // ... defaultConfig ...  
  
  flavorDimensions += 'environment' // Dart side nazewnictwo wymiaru  
  
  productFlavors { // Definicja 3 flavorow  
    dev {  
      dimension 'environment'  
      applicationIdSuffix '.dev' // pl.edu.pam.weather.dev  
      versionNameSuffix '-dev'  
      resValue 'string', 'app_name', 'DEV WeatherApp' // Nazwa pod ikoną  
    }  
  }  
}
```

```

staging {
    dimension 'environment'
    applicationIdSuffix '.staging' // pl.edu.pam.weather.staging
    versionNameSuffix '-staging'
    resValue 'string', 'app_name', 'STG WeatherApp'
}
prod {
    dimension 'environment'
    // Brak sufiku - bazowy package name: pl.edu.pam.weather
    resValue 'string', 'app_name', 'WeatherApp'
}
}

buildTypes {
    release {
        signingConfig signingConfigs.release // Omowione w sekcji 3
        minifyEnabled true // ProGuard / R8 - zaciemnianie kodu
        shrinkResources true // Usuniecie nieuzywanych zasobow
        proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
            'proguard-rules.pro'
    }
}
}

```

2.4. Uruchamianie z flavorem

Uruchamianie konkretnego flavoru (w terminalu):

```

flutter run --flavor dev -t lib/main_dev.dart
flutter run --flavor staging -t lib/main_staging.dart
flutter run --flavor prod -t lib/main_prod.dart

```

Budowanie APK z flavorami:

```

flutter build apk --flavor dev -t lib/main_dev.dart --debug
flutter build apk --flavor prod -t lib/main_prod.dart --release

```

Budowanie App Bundle (do Play Store):

```

flutter build appbundle --flavor prod -t lib/main_prod.dart --release

```

Skrocony zapis (konfiguracja w pubspec.yaml z flutter_flavorizr):

```

flutter run -t lib/main_dev.dart --flavor dev # alias 'dev'

```

pro tip

Możesz dodać skrypty w pubspec.yaml (sekcja scripts) lub użyć paczki flutter_flavorizr, która automatycznie konfiguruje Gradle, Xcode i ikony dla każdego flavoru. Oszczędza to godziny ręcznej konfiguracji.

2.5. Różne ikony dla flavorow

Podstawowym sygnałem wizualnym flavoru są ikony. Aplikacja dev powinna być łatwa do odróżnienia od prod. Typowo stosuje się banner kolorowy (czerwony dla dev, pomarańczowy dla staging, bez baneru dla prod).

Podejście	Opis / kroki
Reczna podmiana ikon w android/app/src/{flavor}/res/	Skopiuj android/app/src/main/res/ do android/app/src/dev/res/ i podmien ikony. Gradle automatycznie wybierze odpowiedni katalog src/{flavor}/ przy buildowaniu.

flutter_launcher_icons + flutter_flavorizr	Paczki automatyzujące. Generują ikony dla wszystkich flavorów z jednego pliku konfiguracyjnego w pubspec.yaml. Rekomendowane dla nowych projektów.
adaptive icons (Android 8+)	Oddzielny foreground i background. Pozwala na overlay banera DEV/STG na ikonie przez zmianę samego foreground. Wygląda profesjonalnie.

3. Podpisywanie aplikacji, czyli Keystore i upload key

Każda aplikacja opublikowana na Google Play musi być podpisana cyfrowo. Podpis kryptograficzny zapewnia, że aktualizacje aplikacji pochodzą od tego samego dewelopera. Jest to mechanizm bezpieczeństwa chroniący użytkowników przed podmienionymi APK.

Analogia: pieczęć notarialna Podpis cyfrowy aplikacji działa jak pieczęć notarialna na dokumencie. Każdy może zobaczyć dokument (APK), ale nie może dać mu pieczęci notarialnej bez klucza notariusza. Google Play sprawdza pieczęć przy każdej aktualizacji - jeśli jest inna niż przy pierwszym przesłaniu, aktualizacja jest odrzucana.

3.1. Dwa klucze w Google Play App Signing

Od 2021 roku Google Play wymaga korzystania z Play App Signing. W tym modelu funkcjonują dwa klucze: upload key i app signing key.

Klucz	Kto przechowuje	Do czego służy	Co się stanie jeśli zgubisz?
Upload key (klucz przesłania)	Ty (deweloper) / CI/CD	Podpisujesz APK/AAB przed uploadem do Play Console	Można zresetować w Play Console (kontakt z supportem)
App Signing key (klucz aplikacji)	GOOGLE (bezpieczna infrastruktura)	Google podpisuje APK przed dostarczeniem użytkownikowi	Nie można zgubić, bo to Google trzyma, nie Ty

Dlaczego dwa klucze?

To podejście separation of concerns: ty możesz zgubić upload key (można zresetować), ale Google nigdy nie zgubi app signing key. Aplikacja dla użytkowników jest zawsze podpisana kluczem app signing. Ty podpisujesz 'kopertę' do Google, a Google przepakowuje apkę z własnym kluczem.

3.2. Generowanie upload keystore

```
# Generowanie keystore (jednorazowe, wykonaj raz na projekt!)
keytool -genkey -v \
  -keystore upload-keystore.jks \      # nazwa pliku keystore
  -keyalg RSA \                        # algorytm RSA (wymagany przez Play)
  -keysize 2048 \                     # rozmiar klucza (minimum 2048 bitów)
  -validity 10000 \                   # ważność: ~27 lat (wystarczy na życie
aplikacji)
  -alias upload                        # alias klucza w keystore

# Narzędzie zapyta o:
# - Hasło do keystore (KEY_STORE_PASSWORD) - ZAPAMIĘTAJ!
# - Hasło do klucza (KEY_PASSWORD) - może być to samo
# - Twoje dane (imie, firma, lokalizacja) - opcjonalne dla upload key

# BEZPIECZENSTWO: NIE commituj upload-keystore.jks do repozytorium!
echo 'upload-keystore.jks' >> .gitignore
```

```
echo 'key.properties' >> .gitignore
```

3.3. Konfiguracja key.properties

Hasła i ścieżka do keystorea trafia do pliku key.properties. Ten plik również NIE powinien być w repozytorium. Jest wstrzykiwany przez CI/CD lub trzymany lokalnie.

```
# android/key.properties - NIE commituj do Git!
storePassword=twoje_haslo_keystorea
keyPassword=twoje_haslo_klucza
keyAlias=upload
storeFile=../upload-keystore.jks # Ścieżka względna od android/app/
```

```
// android/app/build.gradle - wczytaj key.properties i używaj do podpisywania
def keystoreProperties = new Properties()
def keystorePropertiesFile = rootProject.file('key.properties')
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile keystoreProperties['storeFile'] ?
                file(keystoreProperties['storeFile']) : null
            storePassword keystoreProperties['storePassword']
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release // Użyj powyższej konfiguracji
            // ... minifyEnabled, proguardFiles ...
        }
    }
}
```

Kto ma słabą pamięć, ten robi dwa razy

Jeśli zapomnisz dodać signingConfig do buildType release, flutter build appbundle wygeneruje AAB podpisany kluczem debug. Play Console odmowi uploadu. Sprawdź zawsze czy key.properties jest dostępny.

4. Budowanie artefaktów; APK vs App Bundle

Flutter może zbudować dwa rodzaje artefaktów: APK (Android Package) i AAB (Android App Bundle). Różnią się przeznaczeniem, rozmiarem i sposobem dystrybucji.

Cecha	APK (.apk)	App Bundle (.aab)
Przeznaczenie	Bezpośrednia instalacja, dystrybucja poza Play Store, testy	Przesłanie do Google Play Store (od 2021)
Rozmiar na dysku	Zawiera wszystkie architektury	Zawiera tylko metadane; Play generuje optymalne APK per urządzenie
Rozmiar u użytkownika	Pełny rozmiar, bo pobiera się wszystko	Do 40% mniejszy - użytkownik pobiera tylko to, czego potrzebuje
Weryfikacja podpisu	Możesz zainstalować lokalnie, bez sklepu Play (ale tylko do 2027)	Google weryfikuje i podpisuje app signing key
Komenda	flutter build apk --release	flutter build appbundle --release

4.1. Budowanie release z flavor-em

```
# Pełne polecenie budowania App Bundle dla floru prod:
flutter build appbundle \
  --flavor prod \                # który flavor
  -t lib/main_prod.dart \        # punkt wejścia
  --release \                    # tryb release (optymalizacje)
  --obfuscate \                 # zaciemnianie kodu Dart
  --split-debug-info=build/debug-info/ \ # symboliki do kraw stosow
  --tree-shake-icons            # usuniecie nieuzywanych ikon

# Wynik: build/app/outputs/bundle/prodRelease/app-prod-release.aab

# Budowanie split APK (per ABI) - do testow:
flutter build apk --split-per-abi --flavor prod -t lib/main_prod.dart --release
# Tworzy osobne APK dla arm64-v8a, armeabi-v7a, x86_64
```

0 --obfuscate i --split-debug-info

--obfuscate zaciemnia nazwy klas i funkcji Dart w binarnym kodzie. Utrudnia to reverse engineering, ale sprawia, że stack trace w logach są nieczytelne (MyClass.myMethod staje się a.b.c). --split-debug-info zapisuje mapowanie, które pozwala odkryć oryginalny stack trace. ZAWSZE używaj obu razem i archiwizuj debug-info dla każdej wersji - będziesz ich potrzebować do analizy crash-y w Crashlytics.

4.2. Weryfikacja artefaktu przed uploadem

```
# Sprawdź czy AAB jest poprawnie podpisany:
bundletool validate --bundle=app-prod-release.aab

# Sprawdź podpis APK:
apksigner verify --print-certs app-release.apk

# Testuj AAB lokalnie (bundletool generuje lokalne APK z AAB):
bundletool build-apks --bundle=app-prod-release.aab \
  --output=test-install.apks \
  --ks=upload-keystore.jks \
  --ks-pass=pass:twoje_haslo \
  --ks-key-alias=upload \
  --key-pass=pass:twoje_haslo

bundletool install-apks --apks=test-install.apks # Instalacja na podłączonym urządzeniu
```

5. Od konta dewelopera do publikacji na Google Play Store

Proces publikacji aplikacji w Play Store składa się z kilku etapów. Warto je znać, zanim zaczniesz pisać kod CI/CD. Automatyzacja ma sens tylko wtedy, gdy rozumiesz co jest automatyzowane.

5.1. Cykl życia wersji w Play Store

Etap	Co się dzieje	Dostępność	Czas
------	---------------	------------	------

Internal testing	Szybka dystrybucja do max 100 testerów (e-mail whitelist). Brak recenzji Google. Idealny do testów CI/CD.	Tylko whitelist	Natychmiastowa
Closed testing (alfa)	Dystrybucja do grup testerów Google. Testerzy muszą dołączyć przez link. Brak publicznej recenzji.	Grupy testerów	Kilka minut
Open testing (beta)	Publiczny test - każdy może pobrać wersję beta. Widoczne w Play Store z oznaczeniem Beta.	Publiczna	Do 3 dni (pierwsza)
Production	Pełna publikacja. Można robić staged rollout (np. 10% użytkowników najpierw).	Wszyscy użytkownicy	1-3 dni (recenzja)

Wskazówka; użyj Internal Testing Track

Upload AAB, dodaj swój e-mail jako testera. Nie wymaga pełnej recenzji Google i jest darmowy. Będziesz mógł zainstalować aplikację na telefonie w ciągu minut.

5.2. Konfiguracja aplikacji w Play Console

Przed pierwszym uploadem AAB musisz skonfigurować aplikację w Play Console.

Wymagane elementy:

1. Konto dewelopera Google Play - odpłatne jednorazowo 25 USD (trwały dostęp).
2. Strona główna aplikacji: nazwa, opis krótki (80 zn.) i długi (4000 zn.), grafika (ikona 512x512, feature graphic 1024x500).
3. Polityka prywatności - URL do strony z polityką. WYMAGANA jeśli aplikacja zbiera jakiegokolwiek dane (nawet analityczne).
4. Klasyfikacja zawartości - kreatorem ocen IARC (pytania o treść).
5. Zarządzanie danymi - deklaracja jakie dane zbierasz (Data Safety Form).
6. Zapis w Google Play App Signing - potwierdź że Google będzie zarządzał kluczem podpisującym.

5.3. Automatyzacja przez version code i version name

Każdy upload do Play Store musi mieć wyższy versionCode niż poprzedni. W CI/CD można automatycznie generować versionCode na podstawie numeru buildu.

```
# Automatyczny versionCode w GitHub Actions (omówione w sekcji 6)
# versionCode = numer buildu GitHub (GITHUB_RUN_NUMBER)

# pubspec.yaml - wersja semantyczna:
version: 1.2.0+47 # format: versionName+versionCode
#           ^^^ versionName (widoczny dla użytkownika: 1.2.0)
#           ^^ versionCode (monotoniczny licznik, np. 47)

# Nadpisanie przy budowaniu (bez edycji pubspec.yaml):
flutter build appbundle \
  --build-name=1.2.0 \           # versionName
  --build-number=${GITHUB_RUN_NUMBER} \ # versionCode z CI
  --flavor prod -t lib/main_prod.dart -release
```

6. Automatyzacja potoku, CI/CD z GitHub Actions

CI/CD to skrót od Continuous Integration / Continuous Deployment. CI = automatyczne testowanie każdego commita. CD = automatyczne budowanie i publikowanie aplikacji. GitHub Actions to

platforma CI/CD wbudowana w GitHub - darmowa dla repozytoriów publicznych i do 2000 minut miesięcznie dla prywatnych.

Analogia jako linia produkcyjna w fabryce

Wyobraź sobie fabrykę samochodów. Kiedy pracownik wprowadza zmiany do projektu (commit), linia produkcyjna automatycznie: sprawdza quality control (testy), składa auto (build), maluje je (podpisuje), wysyła do showroomu (Play Store). Nikt nie musi ręcznie klikać 'wyślij' - wszystko dzieje się automatycznie. GitHub Actions jest ta linia produkcyjna.

6.1. Koncepcja pipeline

Pipeline (potok) w GitHub Actions składa się z: triggera (co uruchamia), jobow (grup krokow) i stepow (pojedynczych poleceń). Każdy job biegnie na świeżym serwerze (runner).

Pojęcie	Wyjaśnienie / przykład
Workflow (.yml)	Plik YAML definiujący cały pipeline. Trafia do .github/workflows/. Jeden projekt może mieć wiele workflowow.
Trigger (on:)	Zdarzenie uruchamiające workflow: push na gałąź, pull_request, harmonogram (cron), ręcznie (workflow_dispatch).
Job	Grupa „kroków” biegnących na jednym runnerze. Joby mogą biec równolegle lub sekwencyjnie (needs:).
Step	Pojedynczy krok: użycie gotowej akcji (uses:) lub polecenie shell (run:). Wykonywany w kolejności.
Runner	Wirtualna maszyna (Ubuntu, macOS, Windows). ubuntu-latest dla Androida, macos-latest dla iOS.
Secret	Zaszyfrowana zmienna (hasło, klucz API). Definiowana w Settings > Secrets. Nie widoczna w logach.
Artifact	Plik/katalog zapisany po zakończeniu jobu. Można pobrać lub przekazać do następnego jobu.

6.2. Pełny workflow: test + build + deploy

```
# .github/workflows/ci-cd.yml - pełny pipeline CI/CD dla WeatherApp
name: WeatherApp CI/CD

on:
  push:
    branches: [main, develop]      # Uruchom na push do main i develop
  pull_request:
    branches: [main]               # Uruchom na PR do main
  workflow_dispatch:                # Możliwość ręcznego uruchomienia z UI

env:
  FLUTTER_VERSION: '3.22.0'        # Globalne zmienne środowiskowe
  JAVA_VERSION: '17'               # Przypnij wersję Fluttera!

jobs:
  # == JOB 1: Testy jednostkowe i widgetów ==
  test:
    name: Run Tests
    runs-on: ubuntu-latest
    steps:
      - name: Checkout kodu
        uses: actions/checkout@v4   # Pobierz repozytorium

      - name: Konfiguracja Java
        uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '${{ env.JAVA_VERSION }}' }
```

```
- name: Instalacja Flutter SDK
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '${{ env.FLUTTER_VERSION }}'
    channel: stable
    cache: true # Cache SDK - szybsze kolejne uruchomienia

- name: Pobranie zaleznosci
  run: flutter pub get

- name: Weryfikacja formatowania
  run: dart format --output=none --set-exit-if-changed .

- name: Analiza statyczna (linter)
  run: flutter analyze

- name: Uruchom testy z pokryciem
  run: flutter test --coverage --reporter github

- name: Upload coverage do Codecov
  uses: codecov/codecov-action@v4
  with:
    files: coverage/lcov.info

# == JOB 2: Budowanie i publikacja (tylko main branch) ==
build-and-deploy:
  name: Build & Deploy to Play Store
  runs-on: ubuntu-latest
  needs: test # Uruchom TYLKO jeśli test przeszedł!
  if: github.ref == 'refs/heads/main' # Tylko dla main

  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: '${{ env.JAVA_VERSION }}' }
    - uses: subosito/flutter-action@v2
      with: { flutter-version: '${{ env.FLUTTER_VERSION }}', cache: true }

  # - Odtworzenie keystore z secrets -
  - name: Dekoduj keystore z Base64
    run: |
      echo '${{ secrets.KEYSTORE_BASE64 }}' | base64 --decode > android/upload-keystore.jks
      echo 'storePassword=${{ secrets.KEY_STORE_PASSWORD }}' > android/key.properties
      echo 'keyPassword=${{ secrets.KEY_PASSWORD }}' >> android/key.properties
      echo 'keyAlias=${{ secrets.KEY_ALIAS }}' >> android/key.properties
      echo 'storeFile=./upload-keystore.jks' >> android/key.properties

  - run: flutter pub get

  # - Budowanie App Bundle -
  - name: Build App Bundle (prod)
    run: |
      flutter build appbundle \
        --flavor prod \
        -t lib/main_prod.dart \
        --release \
        --obfuscate \
        --split-debug-info=build/debug-info/ \
        --build-number=${{ github.run_number }} # Auto-inkrementowany versionCode

  # - Zapis artefaktow -
  - name: Uploaduj AAB jako artefakt
    uses: actions/upload-artifact@v4
    with:
      name: app-bundle-${{ github.run_number }}
      path: build/app/outputs/bundle/prodRelease/app-prod-release.aab

  # - Publikacja do Play Store Internal Testing -
  - name: Publikacja w Play Store
    uses: r0adk11/upload-google-play@v1
    with:
```

```
serviceAccountJsonPlainText: ${ secrets.SERVICE_ACCOUNT_JSON }
packageName: pl.edu.pam.weather
releaseFiles: build/app/outputs/bundle/prodRelease/app-prod-release.aab
track: internal # internal / alpha / beta / production
status: completed # completed = od razu dostępny
```

6.3. Konfiguracja sekretów w GitHub

Sekrety to zaszyfrowane zmienne dostępne tylko w kontekście GitHub Actions. Definiujesz je w Settings > Secrets and variables > Actions.

Nazwa sekretu	Wartość / jak uzyskać
KEYSTORE_BASE64	Keystore zakodowany w Base64: <code>base64 -i upload-keystore.jks</code> <code>pbcopy</code> (macOS) / <code>base64 upload-keystore.jks</code> (Linux)
KEY_STORE_PASSWORD	Hasło do keystore (wpisane przy <code>keytool -genkey</code>)
KEY_PASSWORD	Hasło do klucza w keystore (może być to samo co <code>KEY_STORE_PASSWORD</code>)
KEY_ALIAS	Alias klucza: <code>upload</code> (lub inny wybrany przy tworzeniu keystore)
SERVICE_ACCOUNT_JSON	JSON z kluczem konta serwisowego Google Cloud z dostępem do Play Developer API. Patrz Play Console > Setup > API access.

Encoding keystore do Base64. Ale dlaczego Base64 zapytacie?

Keystore to plik binarny, nie można go wkleić bezpośrednio do „sekrety” tekstowego. Rozwiązanie: zakoduj go do Base64 (tekst), wklej jako sekret, a w CI dekoduj z powrotem do pliku binarnego (krok 'Dekoduj keystore z Base64' powyżej).

6.4. Matryca buildu - wiele flavorów równolegle

GitHub Actions pozwala zdefiniować matrycę (matrix) - uruchomienie tego samego jobu z różnym zestawem parametrów. Przydatne gdy chcemy budować dev, staging i prod równolegle.

```
jobs:
  build:
    strategy:
      matrix:
        flavor: [dev, staging, prod] # 3 joby równolegle!
      include:
        - flavor: dev
          main: lib/main_dev.dart
          track: internal
        - flavor: staging
          main: lib/main_staging.dart
          track: alpha
        - flavor: prod
          main: lib/main_prod.dart
          track: internal # Prod też najpierw internal
    runs-on: ubuntu-latest
    steps:
      - run: |
          flutter build appbundle \
            --flavor ${ matrix.flavor } \
            -t ${ matrix.main } \
            --release
```

7. Automatyzacja publikacji z Fastlane

Fastlane to narzędzie open-source do automatyzacji publikacji aplikacji mobilnych. Może działać samodzielnie lub w połączeniu z GitHub Actions. Jego główną przewagą to gotowe 'lane' (ścieżki) dla typowych operacji: testowanie, budowanie, podpisywanie, upload do sklepu.

Analogia jako skrypt vs. narzędzie

Kolację można przygotować sobie ręcznie albo użyć robota kuchennego (czy czegośkolwiek automatycznego, oł założmy burżujskiego termomixa).

Skrypty bash/PowerShell to gotowanie ręcznie. Działa, ale trzeba wszystkiego pilnować.

Fastlane to taki termomix. Masz gotowe, sprawdzone przepisy (*actions*) i tylko wywołujesz je po nazwie.

Mniej błędów, mniej kodu, lepsza czytelność.

7.1. Konfiguracja Fastlane

```
# Instalacja Fastlane (wymagane Ruby):
gem install fastlane

# Inicjalizacja w katalogu projektu Flutter:
cd twój_projekt_flutter/
fastlane init
# Fastlane zapyta o platformę: wybierz 'android'
# Fastlane tworzy katalog fastlane/ z plikami Fastfile i Appfile

# fastlane/Appfile - identyfikacja aplikacji
json_key_file("fastlane/service-account.json") # Klucz Google Play API
package_name("pl.edu.pam.weather")             # Package name prod

# fastlane/Fastfile - definicja lanes (ściezek automatyzacji)
default_platform(:android)

platform :android do

  # LANE: Uruchom testy
  lane :test do
    sh('flutter test --coverage --reporter github')
  end

  # LANE: Budowanie i upload do Internal Testing
  lane :deploy_internal do
    sh('flutter build appbundle --flavor prod -t lib/main_prod.dart --release')
    upload_to_play_store(
      track: 'internal',
      aab: 'build/app/outputs/bundle/prodRelease/app-prod-release.aab',
      skip_upload_metadata: true,    # Nie aktualizuj opisu w Play Store
      skip_upload_images: true,     # Nie aktualizuj grafik
    )
  end

  # LANE: Promuj Internal -> Beta
  lane :promote_to_beta do
    upload_to_play_store_internal_app_sharing(
      # Lub prosta promocja:
    )
    # Albo uzywajac supply (starsza akcja):
    supply(track_promote_to: 'beta', track: 'internal')
  end

  # LANE: Pełna publikacja (dev -> staging -> prod)
  lane :full_deploy do
    test
    deploy_internal
    slack(message: 'Nowy build wyleciał do Internal Testing!')
  end
end
```

```
# Uruchamianie lanes:
fastlane test           # Tylko testy
fastlane deploy_internal # Build + upload
fastlane full_deploy    # Testy + build + upload + notyfikacja
```

Integracja Fastlane w GitHub Actions

Zamiast ręcznie pisać kroki flutter build i upload, wywołaj fastlane deploy_internal w kroku run:.
To czystszy kod CI i jest łatwiejszy do testowania lokalnie (uruchomisz te same kroki co CI).

8. Zarządzanie sekretami i zmiennymi środowiskowymi

Klucze API, hasła do baz danych, klucze Firebase, (itd.) to dane, które nie mogą trafić do repozytorium. Istnieje dlatego kilka warstw zarządzania sekretami w projekcie Flutter.

8.1. Zasady bezpiecznego zarządzania kluczami

- Nigdy nie commituj kluczy do Git. Używaj .gitignore dla plików z kluczami.
- Nie „hardkodu” kluczy w kodzie Dart. Nawet zaciemniony kod można zdeobfuskować (odciemnić).
- Używaj osobnych kluczy dla każdego środowiska (dev/staging/prod). Kompromitacja klucza dev nie dotyka prod.
- Rotuj klucze regularnie. Jeśli klucz wyciekł, unieważnij go i wygeneruj nowy.
- Ogranicz uprawnienia kluczy do minimum. Klucz API powinien mieć dostęp tylko do tego co potrzebuje.

8.2. Dart define; wstrzykiwanie kluczy na etapie budowania

Flutter umożliwia przekazanie zmiennych środowiskowych przez --dart-define. Wartości są wbudowywane w kod binarny w momencie kompilacji. Nie są w plikach tekstowych w repozytorium.

Wstrzykniecie klucza API przez --dart-define

```
flutter run --dart-define=API_KEY=abc123 --dart-define=MAP_KEY=xyz456
```

W kodzie Dart odczyt przez String.fromEnvironment

```
const apiKey = String.fromEnvironment('API_KEY', defaultValue: "");
// 'API_KEY' musi dokładnie odpowiadać nazwie w --dart-define!
// defaultValue jest używane gdy budujemy bez --dart-define (np. unit testy)
```

W GitHub Actions jest przekazanie sekretów przez --dart-define

```
run: |
  flutter build appbundle \
    --dart-define=API_KEY=${{ secrets.PROD_API_KEY }} \
    --dart-define=SENTRY_DSN=${{ secrets.SENTRY_DSN }} \
    --flavor prod -t lib/main_prod.dart --release

// odczyt dart-define z typami lib/core/config/app_config.dart
class AppSecrets {
```

```
// const jest wartością znaną w compile time!
static const String apiKey = String.fromEnvironment('API_KEY');
static const String sentryDsn = String.fromEnvironment('SENTRY_DSN');

// Walidacja już w runtime, bo będzie crash-fast jeśli klucza brakuje w prod
static void validate() {
  assert(apiKey.isNotEmpty, 'API_KEY nie został ustawiony! Użyj --dart-define.');
  if (AppConfig.instance.isProd) {
    assert(sentryDsn.isNotEmpty, 'SENTRY_DSN wymagany w prod!');
  }
}
```

8.3. Plik z kluczami; --dart-define-from-file

Przy wielu kluczach wpisywanie ich w wierszu poleceń staje się niepraktyczne. Flutter obsługuje wczytywanie z pliku JSON.

.env.dev.json to plik z kluczami dev (NIE commituj tego do Git-a!)

```
{
  "API_KEY": "dev-key-abc",
  "SENTRY_DSN": "",
  "ANALYTICS_ENABLED": "false"
}
```

Użycie

```
flutter run --dart-define-from-file=.env.dev.json -t lib/main_dev.dart --flavor dev
```

Upewnij się że pliki .env* są ignorowane (przez .gitignore)

```
*.env.json
.env.*
*.env
```

9. Monitorowanie przez Firebase Crashlytics i Analytics

Aplikacja w produkcji to aplikacja, która będzie się crash-ować. Nie dlatego że jest źle napisana, ale dlatego że użytkowników są miliony, urządzenia są różne, sieć jest zawodna. Firebase Crashlytics pozwala zbierać raporty o awariach i analizować je bez dostępu do urządzenia użytkownika.

9.1. Konfiguracja Firebase

Instalacja FlutterFire CLI (narzędzie do konfiguracji Firebase)

```
dart pub global activate flutterfire_cli
```

Konfiguracja projektu Firebase (wymaga konta Firebase i wcześniejszego zalogowania)

```
flutterfire configure \
  --project=weather-app-prod \
  --platforms=android
```


Efekt to `lib/firebase_options.dart` z konfiguracją dla każdej platformy

pubspec.yaml # zależności Firebase
<pre>dependencies: firebase_core: ^3.3.0 firebase_crashlytics: ^4.1.0 # Raporty crash-y firebase_analytics: ^11.2.0 # Analityka użycia</pre>
main.dart
<pre>void main() async { WidgetsFlutterBinding.ensureInitialized(); await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform, // z firebase_options.dart); // Crashlytics: przechwytuje błędy niezłapane przez Flutter FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError; // Przechwytuje błędy asynchroniczne spoza UI PlatformDispatcher.instance.onError = (error, stack) { FirebaseCrashlytics.instance.recordError(error, stack, fatal: true); return true; }; AppConfig.initProd(); // lub initDev() runApp(const ProviderScope(child: MyApp())); }</pre>

9.2. Logi niestandardowe i atrybuty

Dodawanie kluczowych atrybutów użytkownika do raportów crash-ów
<pre>// Pomaga zrozumieć kontekst awarii (który użytkownik, jaki plan) FirebaseCrashlytics.instance.setUserIdentifier(userId); FirebaseCrashlytics.instance.setCustomKey('flavor', AppConfig.instance.flavor.name); FirebaseCrashlytics.instance.setCustomKey('app_version', packageInfo.version); // Manualne logowanie błędu (nie-fatalnego) try { await weatherService.fetchWeather(city); } catch (e, stack) { // Zaloguj w Crashlytics bez crash-owania aplikacji: FirebaseCrashlytics.instance.recordError(e, stack, fatal: false); // Pokaż błąd użytkownikowi i kontynuuj }</pre>

Warto separować Crashlytics między flavor-ami
Stwórz ODDZIELNE projekty Firebase dla dev/staging/prod. Crash-e z testów nie mieszają się z tymi produkcyjnymi. Konfiguruj przez <code>flutterfire configure --project=weather-app-dev</code> dla dev i <code>--project=weather-app-prod</code> dla prod(ukcji), zapisując różne <code>lib/firebase_options_dev.dart</code> i <code>lib/firebase_options_prod.dart</code>.

10. Zadania do wykonania

Zadania bazują na projekcie **WeatherApp v2** z Flutter Lab 2. Każde zadanie kończy się weryfikacją. wykonaj ja zanim przejdziesz dalej.

Zadanie 1 - Konfiguracja Flavorow

1. Utwórz klasę AppConfig z trzema fabrykami: `initDev()`, `initStaging()`, `initProd()`. Pola: `apiBaseUrl`, `appName`, `enableAnalytics`, `showDebugBanner`. Typ wyliczeniowy `AppFlavor`.
2. Utwórz trzy punkty wejścia: `main_dev.dart`, `main_staging.dart`, `main_prod.dart`. Każdy wywołuje odpowiednią fabrykę AppConfig przed `runApp()`.
3. Skonfiguruj Android Gradle (`android/app/build.gradle`): trzy `productFlavors` (`dev`, `staging`, `prod`) z różnym `applicationIdSuffix` i `versionNameSuffix`.
4. Przetestuj ze możesz uruchomić wszystkie trzy flavy jednocześnie na emulatorze (trzy ikony na ekranie głównym). Użyj różnych nazw aplikacji (`DEV/STG/bez prefixu`).
5. W `WeatherCard` wyświetl aktualne środowisko jako mały badge (np. `[DEV]` w lewym górnym rogu karty), widoczny tylko dla flavorow `dev` i `staging`.

WERYFIKACJA: Screenshot AVD z trzema ikonami `WeatherApp` na ekranie głównym z różnymi nazwami. `flutter run --flavor dev` i `--flavor prod` uruchamiają aplikacje bez błędów.

Zadanie 2 - Podpisywanie i budowanie

6. Wygeneruj upload keystore poleceniem `keytool`. Zapisz w katalogu projektu (nie w `android/app/!`). Dodaj `upload-keystore.jks` i `android/key.properties` do `.gitignore`.
7. Skonfiguruj `android/key.properties` z hasłem, aliasem i ścieżką. Skonfiguruj `android/app/build.gradle`: `signingConfigs.release` z wczytywaniem `key.properties`.
8. Zbuduj podpisany App Bundle dla flavoru `prod`: `flutter build appbundle --flavor prod -t lib/main_prod.dart --release`.
9. Zweryfikuj podpis za pomocą `apksigner` lub `bundletool`. Sprawdź rozmiar wygenerowanego `.aab`.
10. Opcjonalnie: załóż konto Google Play (25 USD) i uploaduj AAB do Internal Testing Track.

WERYFIKACJA: Plik `app-prod-release.aab` istnieje w `build/app/outputs/bundle/prodRelease/`. Komenda `apksigner verify` nie zgłosiła błędów. `.gitignore` zawiera wpisy dla keystore.

Zadanie 3 - Pipeline CI/CD z GitHub Actions

11. Stwórz repozytorium GitHub (publiczne lub prywatne) i wypchaj projekt `WeatherApp`.
12. Stwórz workflow `.github/workflows/ci.yml` uruchamiany na push do dowolnej galezi i na `pull_request` do `main`. Workflow powinien: pobrać kod, zainstalować Flutter, uruchomić flutter analyze i flutter test.
13. Dodaj do repozytorium sekrety: `KEYSTORE_BASE64` (zakodowany keystore), `KEY_STORE_PASSWORD`, `KEY_PASSWORD`, `KEY_ALIAS`. Rozszerz workflow o job `build-release` uruchamiany tylko na `main`, zależy od `test` i buduje podpisany AAB.
14. Workflow powinien zapisywać zbudowany AAB jako GitHub Artifact dostępny do pobrania po zakończeniu buildu (`actions/upload-artifact`).
15. Opcjonalnie: dodaj krok publikacji do Play Store Internal Testing przez `r0adkll/upload-google-play` i sekret `SERVICE_ACCOUNT_JSON`.

WERYFIKACJA: Screenshot z GitHub Actions pokazujący zielony workflow z dwoma jobami: `test` (wszystkie kroki zielone) i `build-release` (AAB widoczny jako artefakt do pobrania).

Zadanie 4 - Sekrety i monitorowanie

16. Zrefaktoryzuj AppConfig tak, aby URL API był wstrzykiwany przez `--dart-define=API_BASE_URL`. Utwórz plik `.env.dev.json` z kluczem `API_BASE_URL` ustawionym na adres Open-Meteo. Dodaj `.env.dev.json` do `.gitignore`.
17. Zaktualizuj workflow GitHub Actions, aby w kroku build przekazywać `--dart-define=API_BASE_URL=${{ secrets.PROD_API_URL }}`. Dodaj sekret `PROD_API_URL` do repozytorium GitHub.
18. Zintegruj Firebase Crashlytics (opcjonalnie bez konta Firebase - wystarczy poprawna konfiguracja w kodzie z komentarzem wyjaśniającym kroki inicjalizacji). Pokaż w kodzie: inicjalizację Crashlytics, przechwycenie `FlutterError.onError`, manualne logowanie błędu w bloku `catch`.
19. Napisz dokument `README.md` (w repozytorium GitHub) opisujący: jak uruchomić każdy flavor lokalnie, jak działa pipeline CI/CD, jak dodać nowy sekret do GitHub Actions.

WERYFIKACJA: `README.md` w repozytorium. Workflow GitHub Actions przekazuje `API_BASE_URL` z sekretu. Kod pokazuje inicjalizację Crashlytics.

11. Kryteria oceniania

11.1. Punktacja zadań

Zadanie	Punkty	Co weryfikuje prowadzący
Zad. 1: Flavors	20 pkt	Trzy flavors działają równocześnie na emulatorze. AppConfig z różnym baseUrl. Badge środowiska w UI.
Zad. 2: Podpisywanie	25 pkt	Poprawnie wygenerowany keystore. Podpisany AAB. .gitignore chroni klucze. Zweryfikowany podpis.
Zad. 3: CI/CD	35 pkt	Dwa job-y (test + build). Zielony pipeline na GitHub. AAB dostępny jako artefakt. Sekrety nie wyświetlają się w logach.
Zad. 4: Sekrety i monitoring	20 pkt	dart-define w kodzie i CI. README z dokumentacją. Crashlytics w kodzie.
RAZEM	100 pkt	

11.2. Skala ocen

Ocena	Punkty / wymagania
5.0	90-100 pkt - Wszystkie zadania. CI/CD z automatycznym uploadem do Play Store. README. dart-define w CI.
4.5	80-89 pkt - Zadania 1-3 kompletne. Artefakt AAB w GitHub Actions. Sekrety poprawnie skonfigurowane.
4.0	70-79 pkt - Zadania 1-2 + podstawowy CI (tylko job test). Podpisany AAB zbudowany lokalnie.
3.5	60-69 pkt - Zadania 1-2. Trzy flavors działają. Podpisany AAB. Brak CI/CD.
3.0	50-59 pkt - Zadanie 1: Trzy flavors działają. AppConfig zaimplementowany.
2.0	0-49 pkt - Projekt nie kompiluje lub flavors nie działają.

12. Najczęstsze błędy i ich rozwiązania

Błąd / objaw	Przyczyna i rozwiązanie
Gradle error: Flavor 'dev' not found	flutter run --flavor dev ale brak definicji w build.gradle. Sprawdź czy productFlavors { dev { ... } } istnieje w android/app/build.gradle. Flavourname musi się dokładnie zgadzać (case-sensitive).
MissingPluginException po dodaniu Firebase	Zmiana w build.gradle lub AndroidManifest.xml wymaga pełnego restartu (nie Hot Reload/Restart). Wykonaj: flutter clean && flutter pub get && flutter run.
Upload AAB odrzucony: 'APK was not signed'	buildType release nie ma signingConfig release lub key.properties jest niedostępny. Sprawdź czy plik key.properties istnieje i czy ścieżka w storeFile jest poprawna.
String.fromEnvironment() zwraca pusty string	Brak --dart-define=NAZWA=WARTOSC przy uruchamianiu. W Android Studio skonfiguruj w Run > Edit Configurations > Additional run args: --dart-define=API_KEY=wartość.
GitHub Actions: Permission denied przy uploading to Play	Konto serwisowe nie ma uprawnień do Play Developer API. W Play Console: Setup > API access > Link Service Account. Przyznaj rolę 'Release Manager' lub 'Admin'.
versionCode nie jest większy niż poprzedni	Probujesz uploadować AAB z tym samym lub niższym build number co poprzedni upload. Użyj --build-number=\${{ github.run_number }} w CI lub ręcznie inkrementuj w pubspec.yaml.
Trzy ikony o tej samej nazwie na emulatorze	Brak resValue 'string', 'app_name', '...' w definicjach flavorów lub AndroidManifest.xml używa hardkodowanej nazwy zamiast @string/app_name.

CI/CD: Keystore nieznaleziony przy budowaniu	Plik keystore jest tworzony w kroku 'Dekoduj keystore' ale sciezka w key.properties jest bledna. Sprawdz ze echo zapisuje do android/key.properties a keystore do katalogu root projektu.
AppConfig.instance rzuca AssertionError	AppConfig.initDev() (lub inna fabryka) nie została wywołana przed dostępem do instance. Sprawdź kolejność w main(): najpierw WidgetsFlutterBinding.ensureInitialized(), potem AppConfig.initXxx(), a na koncu runApp().

Przydatne komendy diagnostyczne

```
# Sprawdź jakie flavy sa skonfigurowane w Gradle:
cd android && ./gradlew tasks | grep -i flavor
# Lista dostępnych urzadzen (dla --device-id):
flutter devices
# Verbose logi budowania (gdzie Gradle sie sypie):
flutter build appbundle --verbose 2>&1 | head -100
# Sprawdź zawartosc keystore:
keytool -list -v -keystore upload-keystore.jks
# Weryfikacja podpisu APK:
apksigner verify --verbose --print-certs app-release.apk
# Symulacja CI lokalnie (podobnie do GitHub Actions):
act push # Uzywa narzedzia 'act' (https://github.com/nektos/act)
# Logi GitHub Actions - pelna historia:
gh run view --log # Uzywa GitHub CLI
```

Instrukcja Laboratoryjna Flutter 3 - Publikacja aplikacji, CI/CD i Flavors
Programowanie Aplikacji Mobilnych | Katedra Informatyki i Automatyki

MobileHub

Następne ćwiczenie: Flutter 4 - Testy end-to-end, Flutter Web i Desktop, Isolates